

Q1. Write a program that takes a list of numbers and prints all prime numbers in the list using loops. Do not use any built-in prime-checking functions.

Ans.

```
# List of numbers
numbers = [2, 4, 5, 7, 9, 11, 15, 17, 20, 23]

print("Prime numbers in the list are:")

for num in numbers:
    if num > 1: # Prime numbers are greater than 1
        is_prime = True

        for i in range(2, num):
            if num % i == 0:
                is_prime = False
                break

        if is_prime:
            print(num)
```

Q2. Write a program that generates a multiplication table (1–10) in matrix format using nested loops.

Ans.

```
print("Multiplication Table (1-10):")

for i in range(1, 11):
    for j in range(1, 11):
        print(i * j, end="\t")

    print()
```

3. Write a user-defined function that accepts a list of integers and returns:

- Count of even numbers
- Sum of odd numbers
- Use loops and conditional statements inside the function.

ANS.

```
def check_numbers(numbers):
```

```
    even_count = 0
```

```
    odd_sum = 0
```

```
    for num in numbers:
```

```
        if num % 2 == 0:
```

```
            even_count += 1
```

```
        else:
```

```
            odd_sum += num
```

```
    return even_count, odd_sum
```

```
# List of integers
```

```
nums = [10, 15, 20, 25, 30, 35]
```

```
even_count, odd_sum = check_numbers(nums)
```

```
print("Count of even numbers:", even_count)
```

```
print("Sum of odd numbers:", odd_sum)
```

4. Create a function that calculates simple interest. Use default arguments for rate and time. Demonstrate calling the function using both positional and keyword arguments.

Ans.

```
# Function to calculate simple interest
```

```
def simple_interest(principal, rate=5, time=2):
```

```
    si = (principal * rate * time) / 100
```

```
    return si
```

```
# Calling using positional arguments
```

```
result1 = simple_interest(10000)
```

```
# Calling using keyword arguments
```

```
result2 = simple_interest(principal=10000, rate=8, time=3)
```

```
print("Simple Interest (Positional):", result1)
```

```
print("Simple Interest (Keyword):", result2)
```

5. Create a Student class with attributes: name, marks. Add a method to calculate grade based on marks.

Ans.

```
class Student:
```

```
    def __init__(self, name, marks):
```

```
        self.name = name
```

```
        self.marks = marks
```

```
    def calculate_grade(self):
```

```
        if self.marks >= 90:
```

```
            return "A"
```

```
elif self.marks >= 75:
```

```
    return "B"
```

```
elif self.marks >= 50:
```

```
    return "C"
```

```
else:
```

```
    return "Fail"
```

```
# Create object
```

```
student1 = Student("Harsh", 82)
```

```
print("Name:", student1.name)
```

```
print("Marks:", student1.marks)
```

```
print("Grade:", student1.calculate_grade())
```

6. Create at least two objects and display their grades.

Ans.

```
class Student:
```

```
    def __init__(self, name, marks):
```

```
        self.name = name
```

```
        self.marks = marks
```

```
    def calculate_grade(self):
```

```
        if self.marks >= 90:
```

```
            return "A"
```

```
        elif self.marks >= 75:
```

```
            return "B"
```

```
elif self.marks >= 50:
```

```
    return "C"
```

```
else:
```

```
    return "Fail"
```

```
# Create two objects
```

```
student1 = Student("Harsh", 92)
```

```
student2 = Student("Rahul", 68)
```

```
# Display details and grades
```

```
print("Name:", student1.name)
```

```
print("Grade:", student1.calculate_grade())
```

```
print("Name:", student2.name)
```

```
print("Grade:", student2.calculate_grade())
```

7. Create a BankAccount class with private attributes for balance. Provide methods to deposit and withdraw money. Ensure balance cannot be accessed directly.

Ans.

```
class BankAccount:
```

```
    def __init__(self, balance):
```

```
        self.__balance = balance # Private attribute
```

```
    def deposit(self, amount):
```

```

        self.__balance += amount

    print("Deposited:", amount)

    def withdraw(self, amount):
        if amount <= self.__balance:
            self.__balance -= amount
            print("Withdrawn:", amount)
        else:
            print("Insufficient Balance")

    def show_balance(self):
        print("Current Balance:", self.__balance)

# Create object
account = BankAccount(5000)

account.deposit(2000)
account.withdraw(1000)
account.show_balance()

```

8. Write a program that takes two numbers from the user and performs division.

- Handle the following exceptions:
- Division by zero
- Invalid input type

Ans.

```
try:
```

```
num1 = int(input("Enter first number: "))
```

```
num2 = int(input("Enter second number: "))
```

```
result = num1 / num2
```

```
print("Result:", result)
```

```
except ZeroDivisionError:
```

```
    print("Error: Division by zero is not allowed.")
```

```
except ValueError:
```

```
    print("Error: Invalid input. Please enter numbers only.")
```

9. Write a program that:

- Creates a file
- Writes student data (name and marks)
- Reads and displays the content

Ans.

```
# Create and write data to file
```

```
file = open("student.txt", "w")
```

```
file.write("Name: Harsh\n")
```

```
file.write("Marks: 85\n")
```

```
file.close()
```

```
# Read and display file content
```

```
file = open("student.txt", "r")
```

```
content = file.read()
```

```
print("Student Data:")
```

```
print(content)
```

```
file.close()
```

10. Write a program that reads numbers from a file and calculates:

- Total
- Average
- Handle the case if the file does not exist.

Ans.

```
try:
```

```
    file = open("numbers.txt", "r")
```

```
    numbers = []
```

```
    for line in file:
```

```
        numbers.append(int(line.strip()))
```

```
    file.close()
```

```
    total = sum(numbers)
```

```
    average = total / len(numbers)
```

```
    print("Total:", total)
```

```
    print("Average:", average)
```

```
except FileNotFoundError:
```

```
    print("Error: File does not exist.")
```